Universidad Nacional Autónoma de México

Facultad de Ingeniería

Namespaces y Cgroups en la Virtualización Ligera (Contenedores)

Semestre: 2026-2
Materia: Sistemas Operativos
Profesor: Gunnar Eyal Wolf Iszaevich
Grupo: 7
Fecha de entrega: 13 de abril de 2026

EQUIPO:

Nombre	No. Cuenta
Basilio Illescas Andrés	321137135
Santiago Cruz Macedo Samuel	424065465

Índice

1. Introducción **2**
1.1. Contexto: virtualización tradicional vs. virtualización ligera 2

2. Desarrollo **3**
2.1. El reto: aislamiento y control de recursos 3
2.2. Namespaces: aislamiento de procesos 3
2.2.1. Definición y principio de funcionamiento 3
2.2.2. Tipos de namespaces 4
2.3. Cgroups: control de recursos 5
2.3.1. Definición y principio de funcionamiento 5
2.3.2. Recursos controlados 6
2.4. Namespaces + Cgroups = Contenedores 6

3. Ejemplos técnicos **7**
3.1. Ejemplo 1: aislamiento de procesos con PID namespace 7
3.2. Ejemplo 2: limitación de CPU con cgroups 8
3.3. Ejemplo 3: limitación de memoria con cgroups 9

4. Discusión: relevancia en la industria **10**

5. Conclusiones **11**

Referencias **12**

1. Introducción

En los últimos años, la industria del software ha experimentado una transformación radical gracias a los contenedores. Herramientas como Docker y Kubernetes se han convertido en piezas fundamentales del desarrollo moderno, permitiendo desplegar aplicaciones de forma portátil, rápida y reproducible en cualquier entorno. Sin embargo, la pregunta que con frecuencia queda sin respuesta es: *qué ocurre realmente dentro del sistema operativo cuando se lanza un contenedor?*

La respuesta se encuentra en dos mecanismos del kernel de Linux: los namespaces y los cgroups. Estas características, incorporadas al kernel entre 2002 y 2008, son el motor invisible que hace posible la virtualización ligera y, por tanto, la existencia de los contenedores tal como los conocemos hoy.

El presente trabajo y exposición tienen como objetivos: (1) comprender qué son los namespaces y los cgroups, (2) entender cómo resuelven los problemas de aislamiento y control de recursos en sistemas Linux, (3) demostrar su funcionamiento mediante ejemplos técnicos reproducibles con comandos reales, y (4) vincular estos conceptos con la relevancia actual de los contenedores en la computación en la nube y los microservicios.

1.1. Contexto: virtualización tradicional vs. virtualización ligera

Para entender por qué nacieron los contenedores, es necesario contrastarlos con la virtualización tradicional basada en máquinas virtuales (VM). En una VM, cada instancia incluye un sistema operativo completo, con su propio kernel, que se ejecuta sobre un *hipervisor*, una capa de software que abstrae algunas de las funciones del hardware subyacente. Esta arquitectura ofrece un aislamiento muy robusto, pues cada VM es esencialmente una computadora independiente y la mayor parte de las instrucciones y accesos se ejecutan nativamente; sin embargo, tiene un costo elevado: cada VM define un consumo *estático* de RAM (es decir, si se le asignan 4 GB, los ocupará desde que inicia hasta que termina su ejecución, independientemente de si sus procesos internos los utilizan o no), y además duplica el sistema operativo en cada instancia [1].

Una analogía útil: imaginemos que el servidor es un edificio. Una máquina virtual sería como construir una casa completamente nueva dentro del terreno, con cimientos propios. Un contenedor, en cambio, es como habilitar departamentos dentro del mismo edificio: cada uno tiene su propia puerta, cocina e independencia, pero todos comparten la misma estructura base que sería el kernel del sistema operativo.

Los contenedores Linux operan bajo esta misma analogía. Un contenedor es un conjunto de procesos que se ejecuta directamente sobre el kernel del sistema anfitrión, aislado mediante namespaces y limitado en recursos mediante cgroups. El resultado es un esquema de uso de recursos más eficiente que la virtualización tradicional: los contenedores arrancan en fracciones de segundo y, aunque pueden llegar a ocupar gigabytes, lo hacen de forma dinámica (solo cuando los procesos que los conforman lo requieren), pudiendo

tener un consumo efectivamente nulo en momentos de inactividad. Esto permite ejecutar decenas o cientos de aplicaciones aisladas en un mismo servidor.

2. Desarrollo

2.1. El reto: aislamiento y control de recursos

En un sistema Linux estándar, todos los procesos comparten el mismo kernel y los mismos recursos globales (como los ciclos de CPU, la memoria disponible, las interfaces de red o las operaciones de entrada/salida de disco). Esta arquitectura plantea dos problemas fundamentales en entornos donde múltiples aplicaciones coexisten:

1. **Aislamiento insuficiente.** Sin contención, cualquier proceso puede, en principio, observar y afectar a otros. Un proceso podría leer el árbol de procesos completo del sistema, enviar señales a procesos ajenos o acceder a interfaces de red que no le corresponden. La documentación oficial de Docker advierte al respecto:

   > "Processes running within a container cannot see, and even less affect, processes running in another container or in the host system."
   > (Docker Inc., s.f., *Docker Engine Security*) [2]

   Este aislamiento, que el texto anterior describe como objetivo fundamental, no existe de forma nativa: es precisamente lo que los namespaces deben garantizar.

2. **Competencia sin límites por recursos.** Todos los procesos compiten por CPU, memoria RAM, ancho de banda de disco y red. Un solo proceso con un error de programación puede consumir toda la memoria disponible y provocar que el kernel active el mecanismo *Out-Of-Memory* (OOM), terminando procesos de forma arbitraria o colapsando el servidor (> _ <) [2].

Estos dos retos (el aislamiento y control de recursos) son exactamente los que resuelven los namespaces y los cgroups, respectivamente [1].

2.2. Namespaces: aislamiento de procesos

2.2.1. Definición y principio de funcionamiento

Los namespaces de Linux son un mecanismo del kernel que permite crear entornos aislados para uno o más procesos. En términos precisos, un namespace actúa como un filtro de visibilidad: los procesos dentro de él únicamente perciben los recursos del sistema que están en ese namespace, como si fueran los únicos existentes.

La definición formal la provee la página de manual oficial del kernel de Linux:

> "A namespace wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of the global resource."
> (Kerrisk, 2021, *namespaces(7), Linux manual page*, `https://man7.org/linux/man-pages/man7/namespaces.7.html`) [4]

En otras palabras, el kernel construye una *burbuja* alrededor de un grupo de procesos. Desde dentro de esa burbuja, la visibilidad de los procesos se restringe únicamente a su entorno; desde fuera, el administrador puede ver todo. Este aislamiento es unidireccional: beneficia la seguridad del contenedor sin privar de visibilidad al administrador del host.

Cuando Docker ejecuta un contenedor con `docker run`, crea automáticamente un conjunto de namespaces nuevos para dicho contenedor, garantizando que sus procesos no interfieran con los del host ni con los de otros contenedores [2].

2.2.2. Tipos de namespaces

Linux cuenta actualmente con siete tipos de namespaces, cada uno responsable de aislar un recurso específico [1, 4]:

PID (*Process IDentifier*) namespace. Aísla los identificadores de proceso. Dentro del namespace, los procesos tienen su propia numeración independiente; el primer proceso del contenedor recibe el PID 1, como si fuera el `init` de un sistema independiente. Un proceso dentro del namespace no puede ver ni enviar señales a procesos externos.

Network namespace (NET). Aísla la pila de red: interfaces de red, direcciones IP, tablas de enrutamiento y puertos. Cada contenedor puede tener su propia interfaz `eth0` y escuchar en cualquier puerto sin conflicto con otro contenedor que haga lo mismo.

Mount namespace (MNT). Aísla la vista del sistema de archivos. Un contenedor puede tener su propio directorio raíz (`/`) completamente independiente del host, gracias a técnicas como `chroot` o sistemas de archivos superpuestos como OverlayFS.

UTS (*UNIX Time-Sharing System*) namespace. Aísla el nombre de host (*hostname*) permitiendo que cada contenedor se identifique con un nombre distinto sin modificar el hostname real del servidor.

IPC (*Inter-Process Communication*) namespace. Aísla los recursos de comunicación entre procesos (memoria compartida, semáforos SysV), evitando que un contenedor acceda o bloquee recursos de IPC de otro.

User namespace. Aísla los identificadores de usuario (UID) y grupo (GID). Sobre este tipo, TecnoDigital (2026) lo explica así:

> "podrás tener un `root` dentro que fuera es un usuario sin privilegios."
> (TecnoDigital, 2026, *Namespaces y cgroups en Linux: la base real de los contenedores*, `https://informatecdigital.com/namespaces-y-cgroups-en-linux-la-base-real-de-los-contenedores`) [1]

Esto significa que si alguien lograra escapar del contenedor donde era `root`, no obtendría permisos de superusuario en la máquina real.

Cgroup namespace. Aísla la vista de la jerarquía de cgroups, impidiendo que un proceso dentro de un contenedor conozca la estructura de cgroups del host.

En conjunto, los namespaces proveen el aislamiento básico de los contenedores. TecnoDigital (2026) resume este principio:

> "Un contenedor Linux no es más que un proceso ejecutándose con un conjunto de namespaces nuevos, encajado en uno o varios cgroups y con un sistema de ficheros aislado."
> (TecnoDigital, 2026) [1]

2.3. Cgroups: control de recursos

2.3.1. Definición y principio de funcionamiento

Mientras los namespaces se ocupan de *qué puede ver* un proceso, los cgroups (*control groups*, grupos de control) se ocupan de *cuánto puede usar*. Son una funcionalidad del kernel de Linux que permite organizar procesos en grupos jerárquicos y asignarles límites de consumo de recursos.

La documentación oficial de Red Hat los define con precisión:

> "Control groups are a Linux kernel feature that allows you to allocate resources such as CPU time, system memory, network bandwidth, or combinations of these resources among user-defined groups of tasks."
> (Red Hat, Inc., s.f., *Introduction to Control Groups (Cgroups)*) [3]

En términos prácticos, los cgroups se implementan como una jerarquía de directorios en el sistema de archivos virtual `/sys/fs/cgroup/`. Cada grupo de control es un directorio que contiene archivos especiales donde se leen o asignan los límites: `cpu.max` define la proporción máxima de CPU, `memory.max` establece la cantidad máxima de RAM, y `cgroup.procs` es el archivo donde se inscribe el PID de un proceso para que el kernel le aplique automáticamente las restricciones [5].

2.3.2. Recursos controlados

Los cgroups cuentan con controladores para distintos tipos de recursos [1, 3]:

CPU. Permite asignar una proporción máxima de tiempo de procesador. Un valor de `50000 100000` en `cpu.max` significa que el grupo puede usar 50 ms de cada 100 ms de período, equivalente al 50 % de un núcleo.

Memoria. Establece el consumo máximo de RAM que el grupo puede tener. Si se supera ese límite, el kernel activa el OOM killer, que termina procesos del grupo para recuperar memoria, protegiendo al resto del sistema.

I/O de disco. Limita la velocidad de lectura y escritura en dispositivos de almacenamiento, previniendo que un contenedor acapare todo el ancho de banda de I/O del servidor.

PIDs. Fija un número máximo de procesos o hilos que el grupo puede crear. Esto previene las *fork bombs*, situaciones donde un proceso genera miles de procesos derivados hasta agotar los recursos del sistema.

Docker hace uso explícito de los cgroups. La documentación oficial lo señala:

> "cgroups ensure that each container gets its fair share of CPU, memory, disk I/O, and they also ensure that a single container cannot bring the system down by exhausting one of those resources."
> (Docker Inc., s.f., *Docker Engine Security*) [2]

2.4. Namespaces + Cgroups = Contenedores

La combinación de ambos mecanismos es lo que define a un contenedor. Cuando Docker lanza un contenedor, internamente el runtime (`runc`) realiza los siguientes pasos en el kernel [1]:

1. Crea un conjunto de namespaces nuevos (PID, NET, MNT, UTS, IPC, USER).
2. Inscribe el proceso del contenedor en uno o más cgroups con los límites de recursos especificados.
3. Monta un sistema de archivos aislado (imagen Docker con OverlayFS) como raíz del contenedor.
4. Arranca el proceso principal dentro de ese entorno.

El resultado es un conjunto de procesos que se ejecutan en un entorno lógicamente aislado del resto del sistema, restringidos a utilizar únicamente los recursos que les corresponden y con su propio sistema de archivos. Todo esto ocurre de manera nativa, sin requerir virtualización de hardware ni cargar un kernel adicional, lo que explica la eficiencia característica de los contenedores frente a las máquinas virtuales [1, 2].

3. Ejemplos técnicos

Los siguientes tres experimentos demuestran el funcionamiento real de namespaces y cgroups en un sistema Linux moderno (cgroups v2). Pueden reproducirse en cualquier distribución Linux con permisos de superusuario.

3.1. Ejemplo 1: aislamiento de procesos con PID namespace

Objetivo. Demostrar cómo un namespace de tipo PID aísla la vista de los procesos, de forma que un proceso perciba que es el único en ejecución.

Comandos

Listing 1: Creación de un PID namespace aislado con `unshare`

```
# Crear un nuevo PID namespace con /proc propio
sudo unshare --pid --fork --mount-proc /bin/bash

# Dentro del namespace: listar procesos visibles
ps -aux

# Mostrar el PID de la shell actual dentro del namespace
echo $$

# En otra terminal (host): verificar que el proceso existe
ps -aux | grep bash
```

La utilidad `unshare` es una herramienta de línea de comandos (parte del paquete `util-linux`) que permite ejecutar un programa desvinculando determinados namespaces de los de su proceso padre. En este caso, la opción `--pid` crea el namespace de procesos; `--fork` ejecuta la shell como proceso hijo independiente; y `--mount-proc` monta un `/proc` aislado para que las utilidades del sistema reflejen únicamente los procesos del namespace [1].

Resultado esperado

Al ejecutar `ps -aux` dentro del namespace, la salida mostrará únicamente dos o tres procesos: la shell Bash y el propio `ps`. El comando `echo $$` devolverá `1`, indicando que la shell es el proceso inicial dentro de su espacio aislado. En el host, `ps` mostrará decenas de procesos del sistema, incluyendo la shell aislada con un PID diferente.

Conclusión técnica

El PID namespace aísla el árbol de procesos: los procesos internos no pueden ver ni interactuar con los externos. Esto es exactamente lo que ocurre en cada contenedor

Docker: tiene su propio árbol de procesos independiente del host.

Riesgo o malentendido común

Si se omite `--mount-proc`, herramientas como `ps` continuarán leyendo el `/proc` global, mostrando todos los procesos del sistema y rompiendo el aislamiento. El aislamiento es, además, unidireccional: el administrador del host puede ver y controlar todos los procesos, incluyendo los del interior de cualquier namespace [1].

3.2. Ejemplo 2: limitación de CPU con cgroups

Objetivo. Demostrar que un cgroup de CPU puede restringir el porcentaje de procesador que consume un proceso, incluso si este intenta saturar el núcleo por completo.

Comandos

Listing 2: Limitación de CPU al 50 % mediante cgroups v2

```
# 1. Lanzar un proceso que satura la CPU
yes > /dev/null &

# 2. Verificar en htop que el proceso usa ~100% CPU

# 3. Crear el cgroup y habilitar el controlador de CPU
sudo mkdir /sys/fs/cgroup/limiteCPU
echo "+cpu" | sudo tee /sys/fs/cgroup/cgroup.subtree_control

# 4. Fijar cuota: 50 ms de cada 100 ms = 50% de un nucleo
echo "50000 100000" | sudo tee /sys/fs/cgroup/limiteCPU/cpu.max

# 5. Mover el proceso 'yes' al cgroup
PID_YES=$(pidof yes)
echo $PID_YES | sudo tee /sys/fs/cgroup/limiteCPU/cgroup.procs

# 6. Verificar en htop que el uso de CPU cayo a ~50%
```

Resultado esperado

Antes de aplicar el cgroup, `htop` muestra que `yes` consume aproximadamente el 100 % de un núcleo. Tras inscribirlo en `limiteCPU`, el uso cae a aproximadamente el 50 %, liberando la otra mitad para otros procesos del sistema [1].

Conclusión técnica

El cgroup de CPU actúa como un freno: aunque el proceso intente acaparar el procesador, el kernel lo limita al porcentaje configurado. En entornos con múltiples contenedores, esto garantiza que ninguno pueda monopolizar la CPU y degradar el rendimiento de los demás.

Riesgo o malentendido común

Es importante distinguir entre una cantidad máxima (límite fijo de tiempo de CPU, como el del ejemplo) y las prioridades relativas (*cpu.shares*), que solo actúan cuando hay contención. Fijar una proporción máxima demasiado baja puede hacer que la aplicación responda lentamente; siempre debe ajustarse mediante pruebas.

3.3. Ejemplo 3: limitación de memoria con cgroups

Objetivo. Mostrar cómo un cgroup de memoria puede proteger al sistema de un proceso que intenta consumir más RAM de la permitida.

Comandos

Listing 3: Limitación de RAM a 500 MB mediante cgroups v2

```
# 1. Instalar stress si no esta disponible
sudo apt install stress

# 2. Crear el cgroup y habilitar el controlador de memoria
sudo mkdir /sys/fs/cgroup/limiteMem
echo "+memory" | sudo tee /sys/fs/cgroup/cgroup.subtree_control

# 3. Fijar el limite de RAM en 500 MB
echo "500M" | sudo tee /sys/fs/cgroup/limiteMem/memory.max

# 4. Ejecutar el proceso de carga dentro del cgroup
sudo -u $USER bash -c \
  "echo \$\$ > /sys/fs/cgroup/limiteMem/cgroup.procs && \
   stress --vm 1 --vm-bytes 2000M --timeout 10s"

# 5. Monitorear el consumo real del cgroup
watch -n 1 cat /sys/fs/cgroup/limiteMem/memory.current
```

Resultado esperado

El proceso `stress` intentará reservar 2 GB de RAM; sin embargo, el cgroup lo limitará a aproximadamente 500 MB. El archivo `memory.current` se estabilizará alrededor de 524 288 000 bytes (500 MB). Si el proceso intenta exceder ese límite, el kernel activará el OOM killer únicamente sobre ese proceso, dejando el resto del sistema completamente estable [2].

Conclusión técnica

El cgroup de memoria actúa como un contenedor de seguridad: aunque un proceso tenga un error de fuga de memoria o intente agotar la RAM del servidor, el kernel lo detiene antes de que afecte al resto de las aplicaciones [5].

Riesgo o malentendido común

Si no se limita también la memoria *swap*, el proceso podría alcanzar el tope de RAM y continuar consumiendo espacio de intercambio, degradando el rendimiento. En cgroups v2 se recomienda usar también `memory.swap.max` para un control completo. En cgroups v1, el parámetro equivalente es `memory.limit_in_bytes` [5].

4. Discusión: relevancia en la industria

La comprensión de namespaces y cgroups trasciende el ámbito académico y tiene implicaciones directas en la práctica profesional. Tecnologías como Docker y Kubernetes tienen en estos mecanismos de Linux su fundamento técnico nativo (si bien hoy en día existen reimplementaciones de estas ideas y entornos compatibles para plataformas como Windows o FreeBSD) [1, 2].

En el ámbito de DevOps e integración continua, los contenedores permiten que los mismos artefactos de software se ejecuten de forma idéntica en el entorno de desarrollo, en los servidores de prueba y en producción. En computación en la nube, plataformas como Google Kubernetes Engine o Amazon ECS coordinan miles de contenedores sobre infraestructura compartida, garantizando el aislamiento y la distribución justa de recursos precisamente gracias a namespaces y cgroups. En el modelo de microservicios, cada servicio se despliega en su propio contenedor, lo que facilita la escalabilidad independiente, el despliegue sin interrupciones y la localización de fallos.

Desde la perspectiva de los sistemas operativos, estos mecanismos demuestran cómo el kernel de Linux evolucionó para satisfacer las demandas de la computación moderna: proveer aislamiento sin la sobrecarga de la virtualización de hardware, y gestión de recursos sin sacrificar la eficiencia.

5. Conclusiones

A lo largo de este trabajo se ha demostrado que los contenedores Linux se sustentan en dos mecanismos bien definidos del kernel:

- Los namespaces proporcionan el aislamiento: crean entornos separados dentro de un mismo kernel para que los procesos de distintos contenedores no puedan verse ni afectarse mutuamente. Sin namespaces, no existirían contenedores seguros.
- Los cgroups proporcionan el control de recursos: permiten asignar proporciones máximas de CPU, memoria, I/O y número de procesos a cada contenedor, garantizando un reparto justo y estable. Sin cgroups, un solo contenedor podría colapsar el servidor completo.
- La combinación de ambos mecanismos, más un sistema de archivos superpuesto, es lo que define a un contenedor: un conjunto de procesos aislados, limitados en recursos por el administrador y con su propio entorno de ejecución, gestionado directamente por el kernel sin necesidad de virtualización de hardware.

Los tres ejemplos técnicos presentados hacen evidente que estos conceptos no son abstractos: pueden observarse, medirse y configurarse en cualquier sistema Linux con herramientas estándar.

Comprender namespaces y cgroups es indispensable para cualquier profesional que trabaje con infraestructura moderna. La expansión de Docker, Kubernetes y la computación en la nube hace que estos mecanismos sean más relevantes que nunca en la industria actual [1, 2, 3].

Referencias

[1] TecnoDigital. (2026, 24 de febrero). *Namespaces y cgroups en Linux: la base real de los contenedores*. Informatec Digital. Recuperado de https://informatecdigital.com/namespaces-y-cgroups-en-linux-la-base-real-de-los-contenedores

[2] Docker Inc. (s.f.). *Docker engine security*. Docker Documentation. Recuperado de https://docs.docker.com/engine/security/

[3] Red Hat, Inc. (s.f.). *Chapter 1: Introduction to control groups (cgroups)*. En *Red Hat Enterprise Linux 7 – Resource Management Guide*. Red Hat. Recuperado de https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/resource_management_guide/chap-introduction_to_control_groups

[4] Kerrisk, M. (2021). *namespaces(7) – Linux manual page*. man7.org. Recuperado de https://man7.org/linux/man-pages/man7/namespaces.7.html

[5] Heo, T. (2015). *Control group v2*. The Linux Kernel Documentation. Recuperado de https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html